Práctica Bloque II

"Fusión de Expertos LoRA para Clasificación Multidominio"

Extensiones de ML

Curso: 2025/2026

Rubén José Tenza Martín

Fernando Piñera Hervás

ÍNDICE

Este documento no tiene encabezados. Para agregar encabezados a la tabla de contenido, vaya a Inicio > Estilos

RESUMEN

Este proyecto se fundamenta en la necesidad crítica de especializar los modelos de lenguaje de gran escala para entornos técnicos donde el lenguaje general no es suficiente. El objetivo central ha sido el desarrollo y la evaluación de una arquitectura basada en la fusión de expertos utilizando la técnica de adaptación de bajo rango, conocida como LoRA, aplicada sobre un modelo base BERT. En lugar de seguir el camino tradicional del ajuste fino completo, que consume una cantidad ingente de recursos computacionales, se optó por una estructura modular que permite entrenar adaptadores livianos para dominios específicos, en este caso el médico y el legal, para posteriormente combinar sus conocimientos matemáticamente.

El punto de partida fue un análisis descriptivo profundo de dos conjuntos de datos con naturalezas radicalmente opuestas. El corpus médico, extraído de transcripciones de informes reales, se caracteriza por textos relativamente breves y una terminología técnica pero muy estructurada, lo que facilita que el modelo capture patrones semánticos con rapidez. Por el contrario, el conjunto de datos legal presentó el mayor desafío del proyecto debido a la extensión extrema de sus documentos, que en muchos casos superan las veinte mil palabras, y a una densidad léxica que genera fronteras difusas entre las distintas categorías de clasificación jurídica.

Para gestionar estas diferencias, se implementó una fase de preparación de datos que incluyó el submuestreo para equilibrar las clases y una estrategia de fragmentación o chunking. Dado que BERT tiene una limitación física de quinientos doce tokens, fue necesario seleccionar ventanas de texto representativas de los documentos legales para asegurar que la esencia del mensaje jurídico no se perdiera durante el proceso de tokenización. Esta preparación permitió que el entrenamiento posterior fuera equitativo y que el modelo no desarrollara sesgos hacia el dominio con mayor volumen de palabras.

La metodología se dividió en dos etapas fundamentales, comenzando con una fase de preentrenamiento adaptativo mediante el modelado de lenguaje enmascarado. En esta etapa, se entrenaron dos expertos independientes: uno especializado en medicina y otro en derecho. Cada experto, a través de su propio adaptador LoRA, aprendió los matices, acrónimos y estructuras gramaticales de su área específica. Este enfoque modular permite que el modelo base permanezca congelado, entrenando únicamente una fracción mínima de los parámetros, lo que garantiza que el conocimiento general de BERT no se degrade mientras se adquiere la especialización técnica.

Una vez obtenidos los expertos, se procedió a la comparación entre el modelo de fusión y un modelo generalista. El modelo de fusión se creó promediando aritméticamente los pesos de los dos adaptadores especializados, buscando un equilibrio simétrico entre ambos dominios. Por su parte, el modelo generalista se entrenó utilizando un corpus mixto desde el inicio, sirviendo como grupo de control para validar si la especialización previa aportaba alguna ventaja real. Esta comparativa es la pieza angular de la investigación, pues busca determinar si el conocimiento fragmentado y luego reunido es superior al aprendizaje simultáneo.

Los resultados obtenidos a través de la visualización de embeddings mediante T-SNE confirmaron que la arquitectura de expertos logra una organización del espacio latente muy superior. En las proyecciones vectoriales se observa una separación geográfica clara entre el dominio médico y el legal, lo que demuestra que el modelo ha aprendido a distinguir los contextos con total nitidez. Mientras que el bloque médico muestra agrupaciones densas y compactas, el bloque legal presenta una mayor dispersión, confirmando que la complejidad intrínseca de la jurisprudencia sigue siendo un reto incluso para los modelos especializados.

En cuanto al rendimiento predictivo en la fase de test, el modelo de fusión de expertos logró superar al modelo generalista en la métrica de F1-Score ponderado. Aunque la diferencia numérica parece sutil, es un indicador potente de que la fusión aritmética preserva mejor los matices semánticos que el entrenamiento conjunto, donde a menudo un dominio puede actuar como ruido para el otro. El éxito es especialmente notable en el área médica, donde se alcanzaron niveles de precisión muy elevados, mientras que en el área legal se identificó una oportunidad de mejora relacionada con la gestión de textos de larga duración.

Desde el punto de vista de la eficiencia, el uso de adaptadores LoRA resultó ser un éxito absoluto, manteniendo el consumo de memoria de vídeo por debajo de los siete gigabytes durante todo el proceso. Esto demuestra que es posible alcanzar un alto rendimiento en tareas multidominio utilizando hardware accesible y tiempos de entrenamiento reducidos. Además, la arquitectura de fusión ofrece una ventaja logística inigualable: permite la paralelización del entrenamiento de expertos y facilita la escalabilidad, ya que añadir un nuevo dominio en el futuro no requeriría re-entrenar todo el sistema, sino simplemente añadir un nuevo módulo especializado.

Como conclusión, el proyecto valida la hipótesis de que la modularidad es el camino hacia una inteligencia artificial más sostenible y precisa en entornos técnicos. La capacidad de especializar componentes y luego integrarlos sin pérdida de información abre la puerta a sistemas de clasificación mucho más robustos y fáciles de mantener. Para trabajos futuros, se plantea la integración de modelos con

mayor ventana de contexto para el ámbito legal o la incorporación de técnicas de recuperación de información para asistir al modelo en la clasificación de documentos extremadamente extensos.

1. Definición del problema

1.1 Objetivo del proyecto

El objetivo de este trabajo es:

Evaluar si un modelo de lenguaje entrenado con datos combinados de dos dominios distintos ofrece mejor rendimiento en tareas de clasificación de textos, que dos modelos especializados por dominio, entrenados de forma independiente y posteriormente fusionados.

Para ello, se comparan dos enfoques basados en adaptación al dominio mediante DAPT sobre un modelo BERT congelado con adaptadores LoRA:

1. Un adaptador LoRA entrenado con datos combinados de los dominios médico y legal mediante una fase de preentrenamiento autosupervisado con Masked Language Modeling (MLM).
2. La fusión de dos adaptadores LoRA especializados, entrenados por separado en cada dominio también mediante MLM, y combinados posteriormente en un único adaptador.

Tras la fase de adaptación al dominio, ambos enfoques se emplean en un entrenamiento supervisado downstream para un escenario multitarea de clasificación de textos, permitiendo comparar su comportamiento bajo las mismas condiciones experimentales.

Ambos enfoques se evalúan tanto en términos de rendimiento predictivo como de coste computacional.

1.2 Pregunta de investigación

La pregunta principal que se pretende responder es:

¿Un modelo de lenguaje adaptado a múltiples dominios mediante DAPT conjunto supera, en tareas de clasificación de textos multitarea, a una estrategia basada en DAPT monodominio con posterior fusión de adaptadores especializados?

De forma complementaria, se analizan las siguientes cuestiones:

- ¿Existen diferencias significativas en el consumo de memoria y el tiempo de entrenamiento entre ambos enfoques, tanto en la fase autosupervisada (MLM) como en la fase supervisada?

- ¿Qué estrategia ofrece un mejor equilibrio entre eficiencia computacional y rendimiento en métricas de clasificación?

1.3 Contexto del problema

Los modelos de lenguaje preentrenados como BERT capturan conocimiento general del lenguaje, pero su rendimiento puede degradarse cuando se aplican a dominios altamente especializados, como el médico o el legal, debido a diferencias léxicas, sintácticas y semánticas.

La adaptación al dominio mediante Domain-Adaptive PreTraining (DAPT) permite mitigar este problema a través de una fase adicional de preentrenamiento autosupervisado con datos no etiquetados del dominio objetivo. No obstante, entrenar modelos completamente independientes por dominio no siempre es viable, especialmente en escenarios donde:

- Se requiere soportar múltiples tareas o dominios simultáneamente.
- Existen limitaciones de memoria, tiempo o recursos computacionales.

En este contexto, técnicas de aprendizaje eficiente en parámetros, como LoRA, ofrecen una alternativa escalable al permitir especializar modelos congelados mediante adaptadores ligeros, sin necesidad de reentrenar el conjunto completo de parámetros del modelo base. Sin embargo, no está claro si resulta más eficaz realizar la adaptación al dominio de forma conjunta o de manera independiente seguida de una fusión posterior, especialmente cuando los dominios presentan una notable diferencia semántica.

1.4 Importancia del problema y usuarios potenciales

Este problema es relevante tanto a nivel académico como práctico:

- Investigadores en NLP: interesados en comprender cómo interactúan distintos dominios durante la adaptación al modelo y en escenarios de aprendizaje multitarea.
- Ingenieros de machine learning: que necesitan desplegar modelos multitarea eficientes sin mantener múltiples modelos independientes.
- Aplicaciones industriales: como sistemas de análisis de documentos médico-legales, donde coexisten textos con características lingüísticas muy diferentes.

Los resultados de este estudio pueden ayudar a tomar decisiones informadas sobre estrategias de adaptación al dominio, equilibrando rendimiento, eficiencia y complejidad del sistema.

1.5 Criterios de éxito

Para considerar validada la hipótesis planteada en esta investigación, los resultados deben satisfacer los siguientes criterios:

1. Rendimiento predictivo: mejora o mantiene métricas de clasificación como accuracy y F1-score en ambos dominios.
2. Eficiencia computacional: reduce o mantiene el consumo máximo de memoria (GB) y el tiempo de entrenamiento, tanto en la fase autosupervisada (MLM) como en la fase supervisada.
3. Estabilidad multitarea: evita degradaciones significativas del rendimiento de un dominio al incorporar otro.

1.6 Restricciones del problema

El desarrollo del proyecto está sujeto a las siguientes restricciones:

1. Restricciones de Hardware: Los experimentos se realizan en un entorno limitado a una única GPU NVIDIA T4 (16 GB VRAM) a través de Google Colab. Esto impide realizar un Full Fine-Tuning (ajuste completo de todos los parámetros de BERT), obligando al uso de técnicas de Parameter-Efficient Fine-Tuning (PEFT) como LoRA. Asimismo, limita el tamaño del lote (batch size) a 16 y la longitud de secuencia máxima a 512 tokens.
2. Limitación de Arquitectura (Encoder-Only): El estudio se centra exclusivamente en modelos tipo Encoder (BERT-base-uncased) para tareas de clasificación discriminativa. No se evalúan arquitecturas generativas (GPT, Llama) ni modelos Encoder-Decoder (T5), por lo que las conclusiones son aplicables a tareas de clasificación, pero no necesariamente a generación de texto.
3. Disponibilidad de datos, restringida a dos dominios específicos (médico y legal).
4. Método de Fusión Estática: Este trabajo evalúa la fusión lineal aritmética de pesos (promedio ponderado o suma). No se exploran técnicas de fusión

dinámica (routing o Mixture of Experts) que aprendan a activar expertos en tiempo de inferencia, dejando esto como una línea de trabajo futuro.

2. Análisis Descriptivo

Con el objetivo de evaluar si un modelo de lenguaje entrenado con datos combinados de dos dominios distintos ofrece mejor rendimiento en tareas de clasificación de textos que dos modelos especializados entrenados por separado y posteriormente fusionados, es necesario analizar las características de los datasets que utilizaremos durante las distintas fases del proceso de entrenamiento.

Para este trabajo hemos seleccionado dos conjuntos de datos pertenecientes a ámbitos semánticos muy diferentes: el **dominio médico** y el **dominio legal**. Esta diferencia temática es fundamental, ya que trabajar con textos que no comparten vocabulario y estilo de discurso permite poner a prueba la capacidad real del modelo para generalizar y aprender patrones del lenguaje que no dependen de un único contexto.

Ambos datasets serán utilizados en **dos fases de entrenamiento diferenciadas**:

1. **Entrenamiento autosupervisado:**

   En esta fase, solo utilizaremos los textos completos de cada dataset, sin considerar sus etiquetas. El objetivo es que el modelo aprenda representaciones del lenguaje (embeddings) propias de cada dominio, capturando patrones sintácticos y semánticos relevantes.

2. **Entrenamiento supervisado:**

   A partir del aprendizaje contextual anterior, ajustamos los modelos para la tarea de clasificación multitarea utilizando las etiquetas disponibles en cada conjunto.

Es por ello que el análisis descriptivo se realiza como si ambos datasets fueran puramente de clasificación, ya que, aunque el autosupervisado no requiere un tratamiento especial más allá de la recopilación de textos, la fase supervisada sí exige comprender las clases, su distribución y su significado.

Por otro lado, para garantizar un entrenamiento equilibrado entre los dos dominios, hemos submuestreado 4000 ejemplos de cada dataset, obteniendo así un tamaño homogéneo y comparable.

En resumen, trabajaremos con dos datasets pertenecientes a dominios contrastados (médico y legal), utilizando cada uno tanto en una fase de preentrenamiento autosupervisado como en una fase posterior de clasificación supervisada.

El análisis descriptivo que desarrollaremos a continuación se centrará en presentar sus características, distribución de etiquetas y propiedades relevantes para entrenar modelos de clasificación de texto, tomando como base un total de 4000 muestras por dominio. Esto nos permitirá evaluar si el entrenamiento conjunto en ambos ámbitos beneficia la calidad y eficiencia del modelo frente al enfoque basado en modelos especializados por separado.

2.1 Descripción del Dataset Médico

El primer conjunto de datos procede de HuggingFace: **rungalileo / medical_transcription_4**, un dataset basado en transcripciones médicas y reportes clínicos. Contiene textos médicos acompañados de **cuatro etiquetas** que clasifican el tipo de documento clínico:

1. **Medical Records:** Incluye registros médicos generales, como informes de evolución, revisiones clínicas o anotaciones básicas.
2. **Other:** Textos relacionados con el ámbito médico que no encajan claramente en las categorías principales (por ejemplo, notas administrativas).
3. **Internal Medicine:** Documentos relacionados con medicina interna, como diagnósticos, evaluaciones clínicas y tratamientos no quirúrgicos.
4. **Surgery:** Informes y transcripciones vinculados a procedimientos quirúrgicos, intervenciones, técnicas operatorias o evoluciones postquirúrgicas.

El dataset cuenta originalmente con **4.499 ejemplos de entrenamiento** y **500 de prueba**. Su formato incluye tres campos principales:

1. **id:** identificador numérico.
2. **text:** la transcripción médica.
3. **label:** la categoría clínica.

A continuación, mostramos un breve resumen visual de este dataset con varios ejemplos:

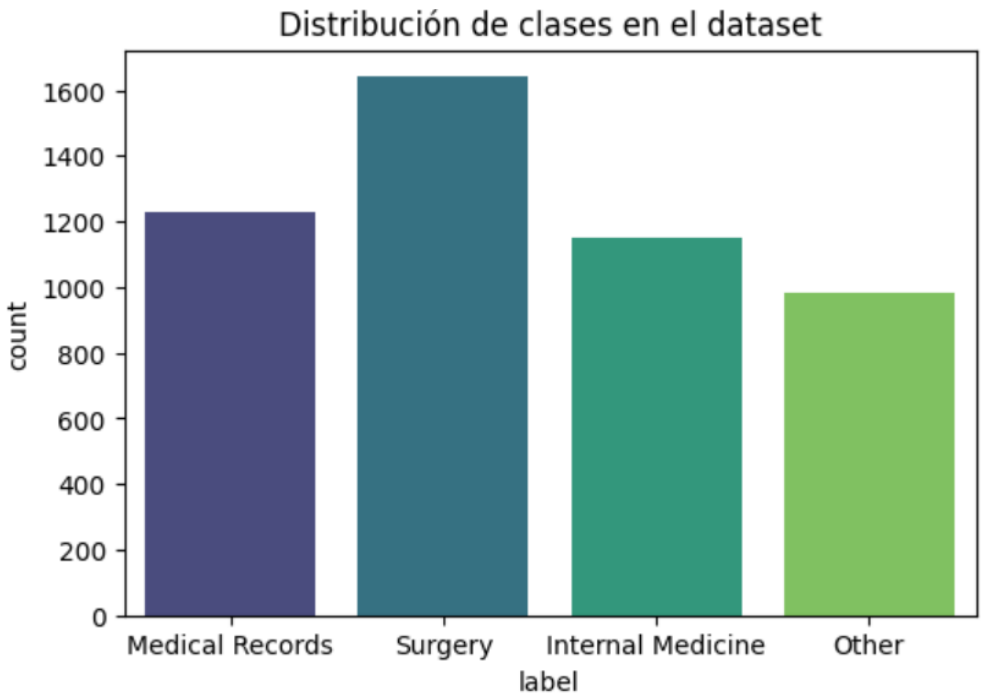
```
Tamaño total del dataset: 4999

Distribución por partición:
Train: 4499  Test: 500

Ejemplo de registros:
                                                   text            label
84    HISTORY: , This 57-year-old female who present...  Medical Records
2470  EXAM: , Cervical, lumbosacral, thoracic spine ...            Other
2804  PREOPERATIVE DIAGNOSIS: , Internal derangement...            Other
4987  PREOPERATIVE DIAGNOSES: , Left elbow fracture ...          Surgery
4924  PREOPERATIVE DIAGNOSES:,1.  Intrauterine pregn...          Surgery

Valores nulos por columna:
text     0
label    0
dtype: int64
```

Podemos observar cómo no hay ejemplos no etiquetados o ejemplos con contenido de texto vacío o nulo.

2.1.1 Distribución de clases

Seguidamente, vamos a visualizar cuántos ejemplos pertenecen a cada categoría, permitiendo:

- Detectar desbalances entre clases.
- Entender la importancia relativa de cada categoría.
- Anticipar posibles problemas en el entrenamiento (por ejemplo, clases escasas que dificulten la generalización).



```
Conteo de etiquetas:
label
Surgery              1640
Medical Records      1228
Internal Medicine    1149
Other                 982
Name: count, dtype: int64

Porcentajes:
label
Surgery              32.806561
Medical Records      24.564913
Internal Medicine    22.984597
Other                19.643929
```

Los resultados muestran un desbalance moderado en las clases, tal como el propósito de este apartado buscaba detectar.

- La clase '**Surgery**' es la más frecuente, con 1640 ejemplos (casi el 33% del total).
- La clase '**Other**' es la menos frecuente, con 982 ejemplos (cerca del 20%).
- '**Medical Records**' (24.6%) e '**Internal Medicine**' (22.9%) se sitúan en posiciones intermedias con 1228 y 1149 ejemplos respectivamente.

Esta desigualdad en la distribución implica que el modelo podría tender a aprender mejor las clases más frecuentes, mientras que las menos representadas podrían presentar mayor dificultad de reconocimiento. Por ello, es importante considerar técnicas de balanceo o métricas adecuadas para evitar que el predominio de ciertas etiquetas sesgue el rendimiento global del modelo.

Igualmente, podemos decir que todas las clases tienen una representación suficiente para aprender patrones, y que a pesar del desbalanceo, existe un equilibrio razonable que evita que una única clase domine el aprendizaje.

2.1.2 Longitud de Frases

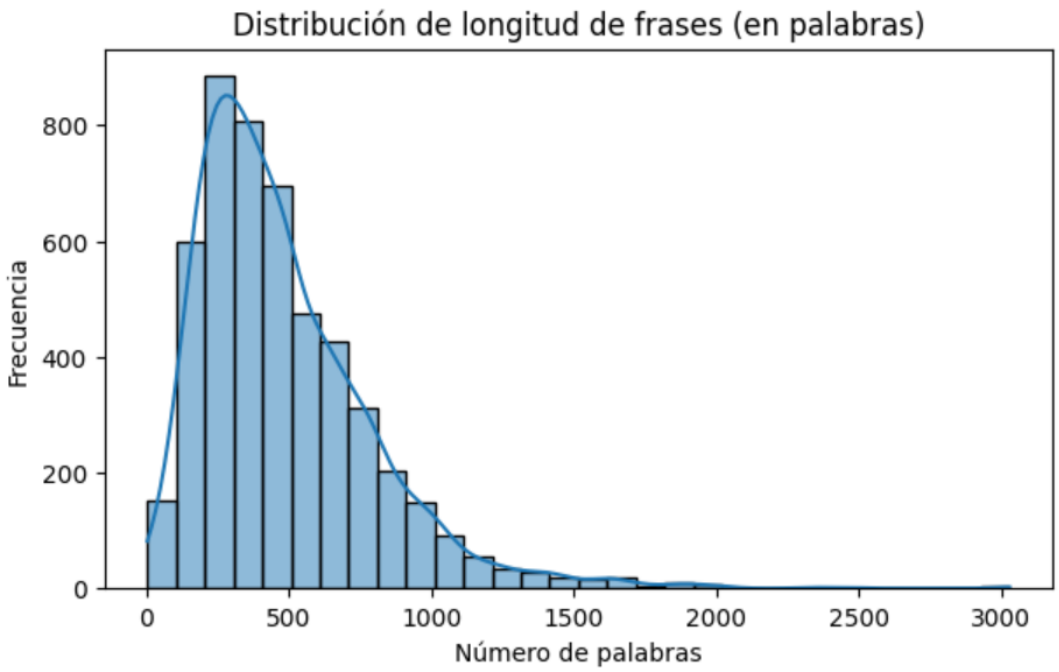
Por otro lado, podemos ver cuántas palabras contiene cada texto y representar la distribución de longitudes.

Así, podemos observar:

- Si los textos son largos o cortos.
- Cuánta variabilidad existe.
- Si el modelo necesitará manejar secuencias extensas.

- Detectar posibles valores atípicos (textos extremadamente largos o muy breves).

Debemos tener en cuenta que ambos datasets deben tener una longitud media de texto más o menos similar, para no hacer que el modelo aprenda esa característica como algo propio de un dominio u otro.




```
Estadísticas de longitud:
count    4999.000000
mean      482.303461
std       313.296265
min         2.000000
25%       260.000000
50%       414.000000
75%       632.000000
max      3029.000000
```

Este dataset tiene un **promedio** de 482 palabras por texto, con una **desviación estándar** de aproximadamente 313 palabras. Esto indica que, aunque la mayoría de los textos se sitúan cerca de la media, existe una gran variabilidad en la longitud, con ejemplos mucho más cortos o mucho más largos que el promedio.

Podemos observar un rango muy amplio: hay textos extremadamente breves y otros muy extensos. La **mediana** (414 palabras) es menor que la media, lo que sugiere que unos pocos textos muy largos elevan el promedio, creando una distribución sesgada hacia la derecha, tal y como vemos en el gráfico.

Implicaciones:

- La gran variabilidad en la longitud significa que el modelo debe ser capaz de manejar secuencias de diferente tamaño.

- Los textos muy largos podrían requerir truncamiento o segmentación para ajustarse a limitaciones de memoria o tamaño de entrada del modelo.
- Los textos cortos podrían aportar poca información, por lo que el modelo deberá aprender a extraer características relevantes incluso de muestras pequeñas.

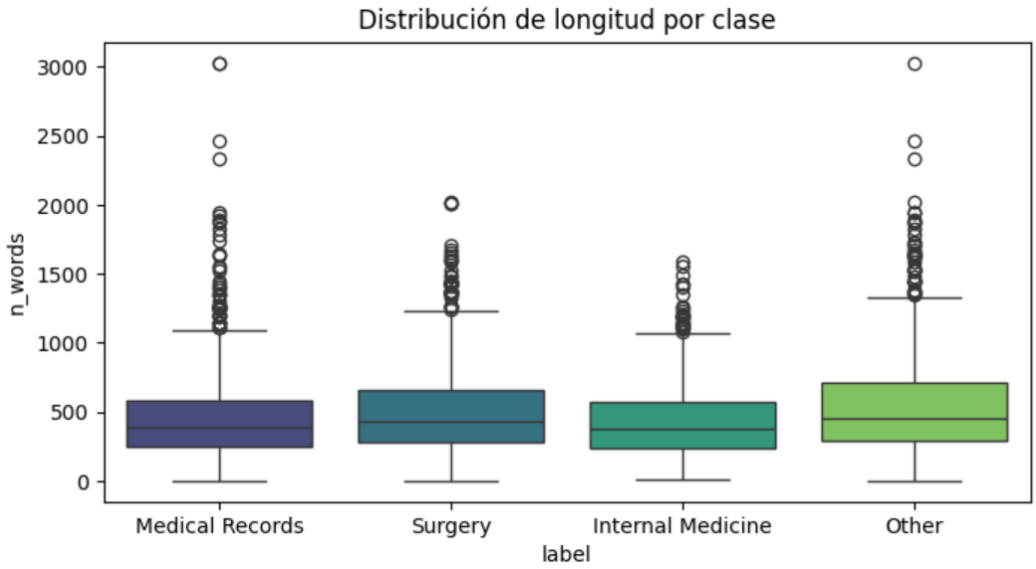
En reusmen, estas estadísticas muestran un dataset con diversidad suficiente para entrenar un modelo que capture patrones semánticos complejos, pero también presentan dificultades relacionados con la gestión de la longitud de las secuencias.

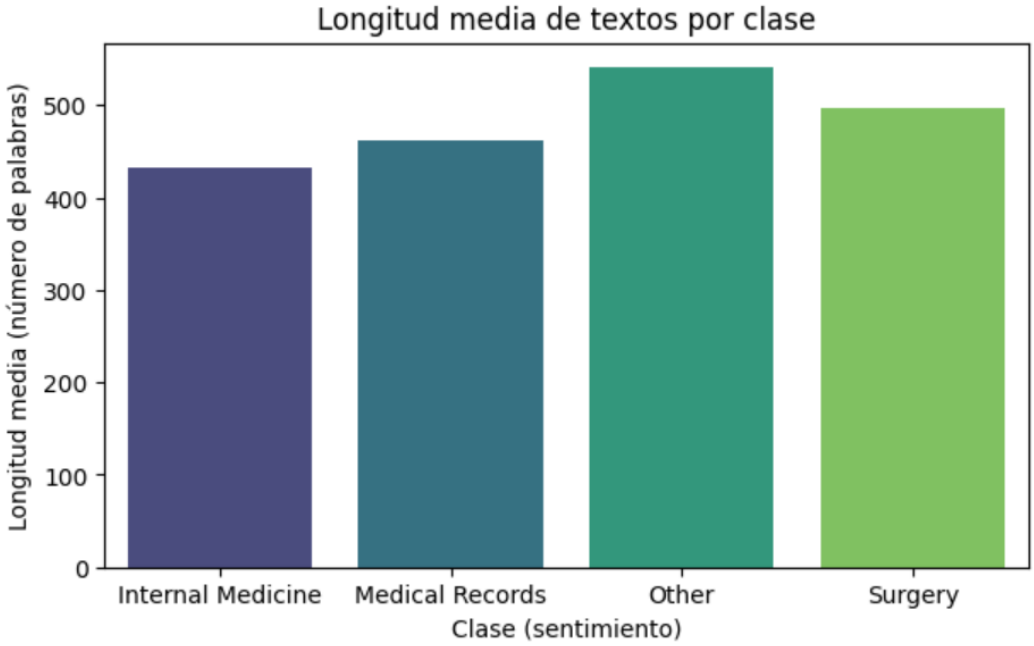
2.1.3 Distribución de Longitud por Clase

En esta parte analizamos si la longitud del texto depende de la clase. Mostraremos gráficos como boxplots y barras para observar:

- Diferencias en la media de longitud entre categorías.
- Dispersión.
- Presencia de outliers particulares por clase.

Esta información es importante porque, si las clases tienen longitudes muy distintas, el modelo podría aprender a clasificar solo por la extensión del texto.






```
                    count        mean         std   min     25%    50%  \
label
Internal Medicine  1149.0  433.296780  263.219808  10.0  237.00  373.0
Medical Records    1228.0  462.402280  320.272082   2.0  251.75  392.5
Other               982.0  540.500000  366.425644   3.0  293.00  454.5
Surgery            1640.0  496.692683  299.023353   2.0  274.75  430.0

                      75%     max
label
Internal Medicine  570.00  1593.0
Medical Records    588.00  3029.0
Other              707.25  3029.0
Surgery            661.00  2019.0
```

Observamos como las clases Other y Surgery tienden a tener textos más largos, mientras que Internal Medicine y Medical Records contienen textos algo más cortos en promedio. La desviación estándar es considerable en todas las clases (entre 263 y 366), lo que refleja una gran variabilidad interna dentro de cada categoría.

Por otro lado, el mínimo de palabras por clase es muy bajo (2–10), lo que evidencia que hay textos extremadamente cortos en todas las categorías.

Además, los percentiles muestran que la mayoría de los textos se concentra en un rango intermedio, aunque hay valores máximos muy altos (hasta 3.029 palabras en Medical Records y Other), lo que provoca una distribución sesgada hacia textos muy largos en algunas clases. Este hecho se puede evidenciar en los boxplots, los cuales revelan que todas las clases tienen numerosos outliers de textos largos.

Implicaciones:

- La variabilidad entre clases podría influir en la capacidad del modelo para generalizar: clases con textos muy largos pueden aportar más contexto y

características, mientras que las más cortas podrían ser más difíciles de clasificar correctamente.

- Nuevamente concluimos que es importante considerar estrategias de preprocesamiento, como truncamiento o padding, para manejar diferencias de longitud y evitar que los textos más extensos dominen el aprendizaje.

Sin embargo, podemos decir que no hay una clara diferencia entre las longitudes medias de cada clase, por lo que no hay un excesivo riesgo de que el modelo clasifique en función a dicha característica.

2.1.4 Palabras más frecuentes por clase

Otro análisis muy interesante es el de palabras más frecuentes por clase, viendo qué palabras aparecen con mayor frecuencia dentro de cada categoría, y analizando su vocabulario característico.

Tras el proceso de limpieza y tokenización correspondiente y haciendo un recuento, observamos como términos como 'patient' (paciente), 'history' (historial), 'right' (derecho), 'left' (izquierdo), 'normal' y 'pain' (dolor) son muy frecuentes en todas las clases. Esto define el vocabulario general del dominio médico.

Por otro lado, podemos observar palabras distintivas a cada clase:

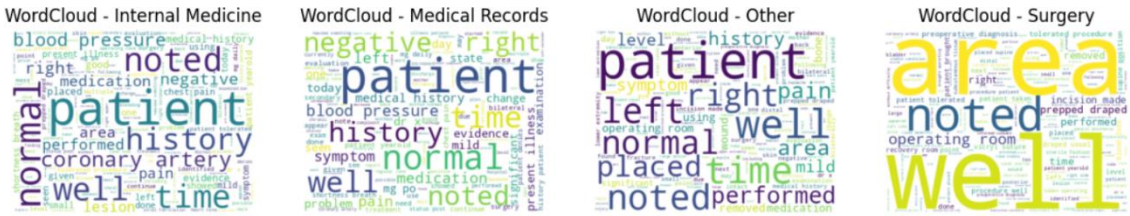
1. **Surgery**: Palabras frecuentes: patient, placed, right, left, procedure, incision, removed, performed, skin, room, diagnosis.
    - **Observación**: Dominada por términos quirúrgicos y procedimentales (incision, removed, procedure, room), mostrando que los textos describen intervenciones, técnicas y entornos operatorios.
2. **Internal Medicine**: Palabras frecuentes: patient, history, left, right, artery, pain, blood, procedure.
    - **Observación**: predominan términos relacionados con diagnósticos internos, anatomía y medidas clínicas (artery, mg, blood), reflejando que se centra en patologías no quirúrgicas.
3. **Medical Records**: Palabras frecuentes: patient, history, normal, pain, mg, blood, time, medical
    - **Observación**: el vocabulario incluye términos más generales y administrativos (time, dr, medical).
4. **Other**: Palabras frecuentes: patient, right, left, placed, pain, procedure, used, noted, performed

- **Observación**: Los textos incluyen vocabulario variado, un mix de procedimientos y observaciones.

Todo esto sugiere que, aunque algunos términos comunes podrían inducir confusión, la presencia de palabras específicas por clase ayuda a que un modelo basado en embeddings aprenda representaciones diferenciadas y útiles para la tarea de clasificación.

2.1.5 Nubes de Palabras

De forma complementaria, observamos de forma más visual lo que habíamos explicado y desarrollado en el anterior apartado, mostrando una nube de palabras donde las palabras más frecuentes aparecen más grandes, ofreciendo así una visualización más intuitiva y rápida del contenido de cada clase.



2.2 Descripción del Dataset Legal

El segundo conjunto procede de Kaggle: **Legal Text Classification Dataset**, recopilado por **A. Mohan Kumar**, con textos procedentes del Australian Legal Information Institute (AustLII). Incluye **25.000 casos legales** de los años 2006 a 2009, cada uno representado mediante:

1. **Case ID:** identificador único.
2. **Case Outcome:** resultado del caso judicial.
3. **Case Title:** título del caso.
4. **Case Text:** el texto completo de la resolución o sentencia.

El conjunto original contiene las siguientes etiquetas (*Case Outcome*), que describen la forma en que un caso judicial utiliza, interpreta o contrasta las decisiones de casos anteriores:

'cited', 'applied', 'followed', 'referred to', 'related', 'considered', 'discussed', 'distinguished', 'affirmed', 'approved'

Con el objetivo de reducir redundancias, mejorar la discriminación entre clases y simplificar el entrenamiento, hemos seleccionado únicamente **tres etiquetas**:

1. **considered:** indica que un caso ha sido evaluado como referencia relevante, sin adoptar necesariamente su decisión.

2. **distinguished:** señala que el caso citado se diferencia del actual por hechos o aspectos legales, justificando por qué no debe aplicarse su precedente.
3. **applied:** indica que los precedentes del caso citado han sido utilizados directamente en la resolución del caso actual.

Esta selección permite mantener categorías con significado jurídico claro y bien diferenciado. De manera conceptual:

- **applied:** relación positiva, aceptación del precedente.
- **distinguished:** relación negativa, diferencias que impiden la aplicación del precedente.
- **considered:** categoría neutra, donde el precedente es tenido en cuenta sin ser aceptado ni rechazado.

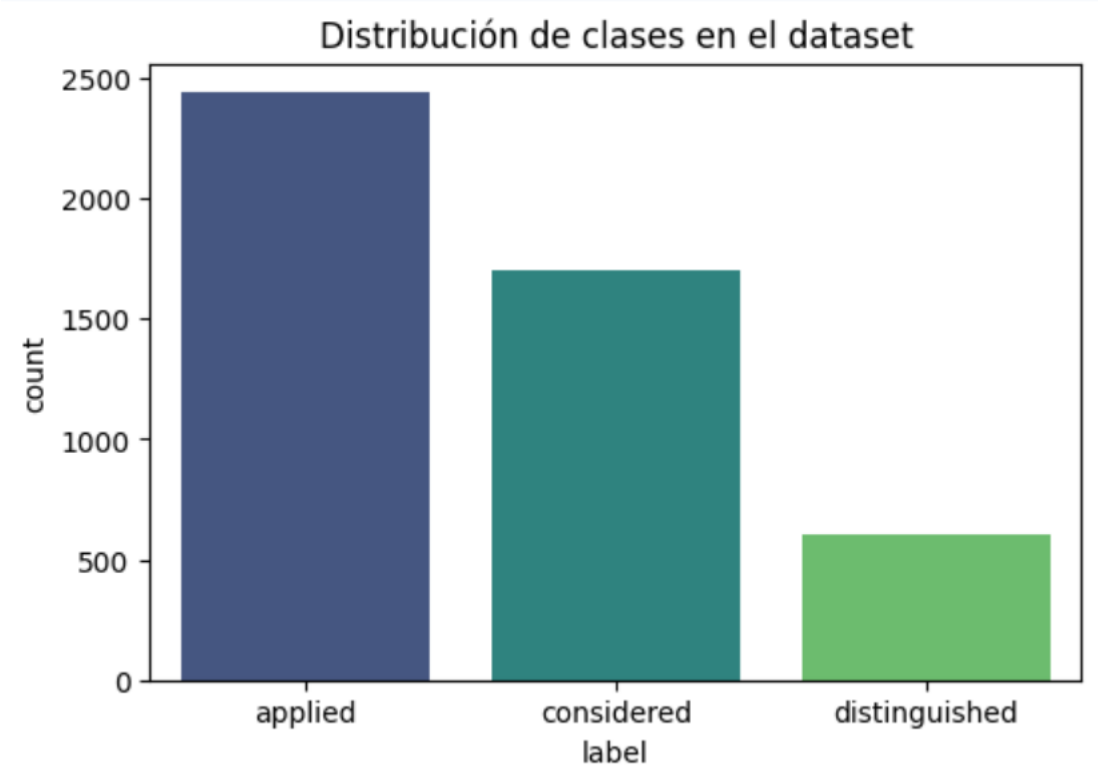
A continuación, mostramos un breve resumen visual de este dataset con varios ejemplos. Para ello eliminamos campos que no aportan información al análisis, como el identificador del caso y el título, estandarizamos el nombre de las columnas de Case Text y Outcome a 'text' y 'label', y filtramos quedándonos con aquellos ejemplos clasificados dentro de las 3 posibles alternativas que hemos contemplado:

```
Tamaño total del dataset: 4768

Ejemplo de registros:
               label                                                text
5110         applied  second ground of appeal is that the appellant ...
11459  distinguished  As I said in Spriggs, argument by analogy must...
13324     considered  Optiver stressed that the question for it to d...
20814     considered  AWB submitted that the policy considerations t...
3372      considered  Section 177D(b) requires the decision-maker to...

Valores nulos por columna:
label     0
text     28
dtype: int64
```

Podemos ver en este resumen como no hay ejemplos sin etiqueta, pero si hay ejemplos nulos, **por lo que procedemos a su eliminación para asegurar la calidad del conjunto de datos.** (No podemos usar otra técnica ya que estamos tratando texto).

2.2.1 Distribución de clases

Seguidamente, vamos a visualizar cuántos ejemplos pertenecen a cada categoría:




```
Conteo de etiquetas:
label
applied          2438
considered       1699
distinguished     603
Name: count, dtype: int64

Porcentajes:
label
applied          51.434599
considered       35.843882
distinguished    12.721519
Name: proportion, dtype: float64
```

Los resultados muestran un desbalance moderado entre las clases, tal como el propósito de este apartado buscaba detectar.

- La clase 'applied' es la más frecuente, con 2438 ejemplos (alrededor del 51% del total).
- La clase 'distinguished' es la menos representada, con 603 ejemplos (cerca del 13%).
- Por su parte, 'considered' se sitúa en una posición intermedia con 1699 ejemplos (aproximadamente 36%).

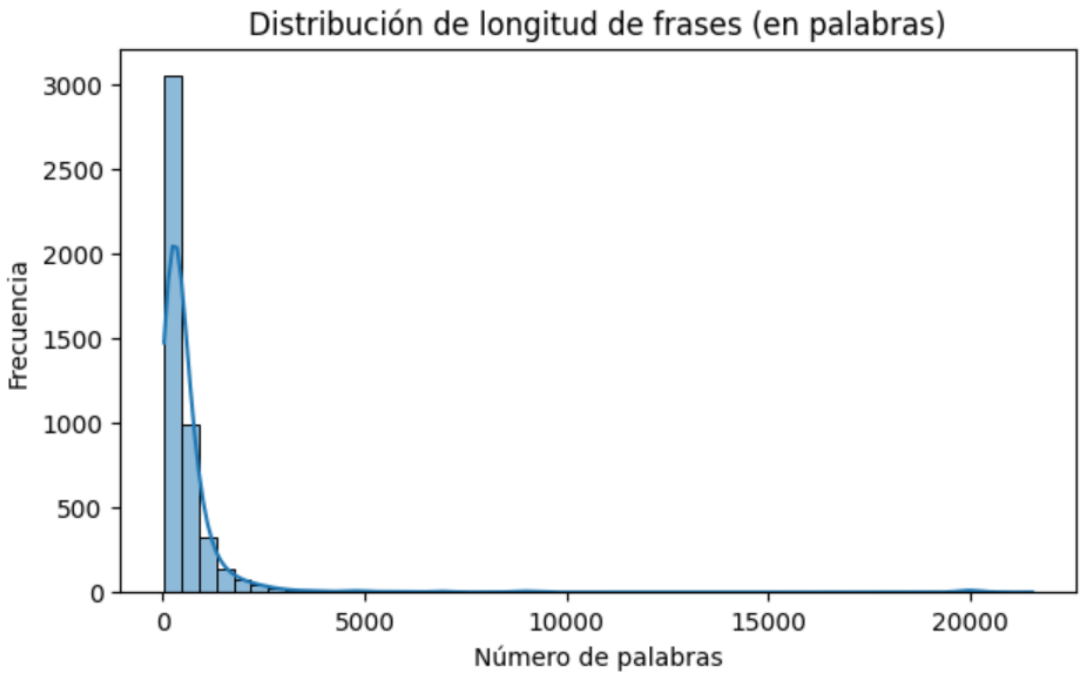
Esta desigualdad en la distribución implica que el modelo podría aprender con mayor facilidad la clase dominante ('applied'), mientras que las clases menos

frecuentes, especialmente 'distinguished', podrían resultar más difíciles de reconocer. Por ello, conviene considerar técnicas de balanceo o el uso de métricas adecuadas para evitar que el sesgo hacia la clase mayoritaria afecte al rendimiento global.

Aun así, todas las clases cuentan con una cantidad razonable de ejemplos, por lo que el modelo podrá aprender patrones significativos sin que una categoría quede totalmente infrarrepresentada.

2.2.2 Longitud de Frases

En este apartado calculamos cuántas palabras contiene cada texto y representamos la distribución de longitudes.




```
Estadísticas de longitud:
count     4740.000000
mean       622.113713
std       1462.134251
min         26.000000
25%        184.000000
50%        330.000000
75%        618.000000
max      21535.000000
Name: n_words, dtype: float64
```

Este dataset tiene un promedio de 622 palabras por texto, con una desviación estándar de aproximadamente 1462 palabras. Esto indica que, aunque muchos textos se sitúan cerca de la media, existe una gran variabilidad en la longitud, con ejemplos mucho más cortos o extremadamente largos que el promedio.

Podemos observar un rango muy amplio: el texto más corto tiene 26 palabras y el más largo 21,535 palabras. La mediana (330 palabras) es considerablemente menor que la media, lo que sugiere que unos pocos textos muy extensos elevan el promedio, generando una distribución sesgada hacia la derecha, tal como se aprecia en el gráfico.

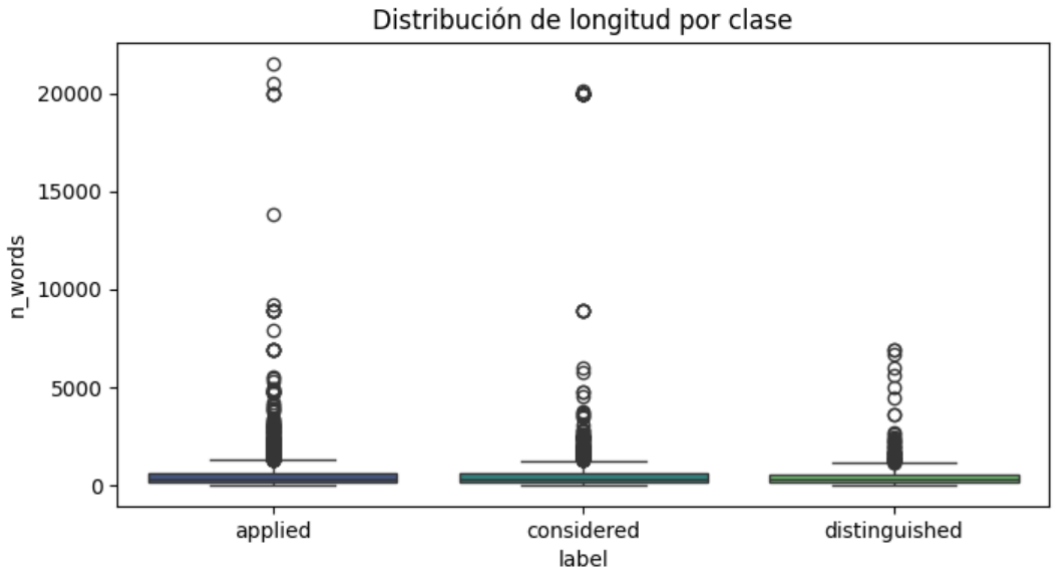
Implicaciones:

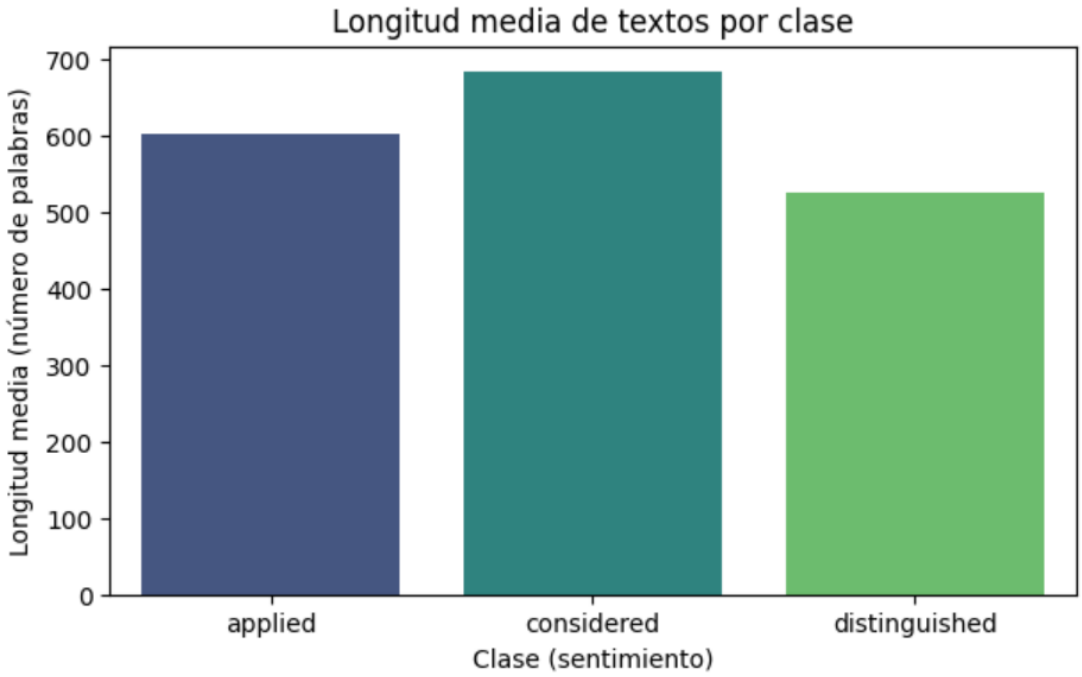
- La gran variabilidad en la longitud significa que el modelo debe manejar secuencias de diferente tamaño.
- Los textos muy largos podrían requerir truncamiento o segmentación para ajustarse a las limitaciones de memoria o al tamaño de entrada del modelo.
- Los textos cortos aportan poca información, por lo que el modelo deberá extraer características relevantes incluso de muestras pequeñas.

En resumen, estas estadísticas muestran un dataset con suficiente diversidad para entrenar un modelo capaz de capturar patrones semánticos complejos, pero también presentan desafíos relacionados con la gestión de la longitud de las secuencias.

2.2.3 Distribución de longitud por clase

Seguidamente, en esta parte vamos a analizar si la longitud del texto depende de la clase, y para ello observamos gráficos como boxplots y barras:






```
                count        mean          std   min    25%    50%    75%  \
label
applied        2438.0  603.485234  1239.827060  26.0  180.0  329.0  619.5
considered     1699.0  682.961742  1884.371861  34.0  188.5  342.0  623.0
distinguished   603.0  525.986733   754.892353  29.0  177.0  313.0  580.0

                   max
label
applied        21535.0
considered     20134.0
distinguished   6924.0
```

Observamos que las clases considered y applied tienden a tener textos más largos en promedio (682 y 603 palabras, respectivamente), mientras que distinguished contiene textos algo más cortos (526 palabras). La desviación estándar es considerable en todas las clases (entre 755 y 1884), lo que refleja una gran variabilidad interna dentro de cada categoría.

Por otro lado, el mínimo de palabras por clase es relativamente bajo (26–34), evidenciando que existen textos muy cortos en todas las categorías. Los percentiles muestran que la mayoría de los textos se concentra en un rango intermedio, aunque los valores máximos son extremadamente altos (hasta 21,535 palabras en applied y 20,134 en considered), lo que provoca una distribución sesgada hacia textos muy largos. Esto se refleja claramente en los boxplots, donde aparecen numerosos outliers de longitud elevada en todas las clases.

Implicaciones:

- La variabilidad entre clases podría influir en la capacidad del modelo para generalizar: textos muy largos aportan más contexto y características, mientras que los más cortos podrían ser más difíciles de clasificar.

- Es importante considerar estrategias de preprocesamiento, como truncamiento o padding, para manejar diferencias de longitud y evitar que los textos más extensos dominen el aprendizaje.

Sin embargo, no hay una diferencia marcada entre las longitudes medias de cada clase, por lo que el riesgo de que el modelo clasifique únicamente por esta característica es bajo.

2.2.4 Palabras más frecuentes por clase

Por último, tal y como hicimos con el dataset médico, calcularemos qué palabras aparecen con mayor frecuencia dentro de cada categoría, y así podremos analizar su vocabulario característico. Para ello, hacemos un recuento de palabras:

Términos como 'v', 'court', 'case', 'act', 'j' y 'decision' aparecen muy frecuentemente en todas las clases. Esto define el vocabulario general del dominio jurídico.

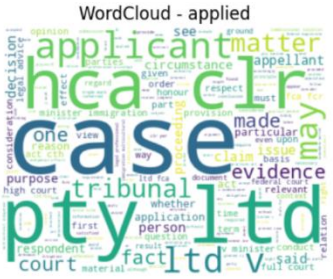
Por otro lado, podemos observar palabras relativamente distintivas en cada clase:

- Applied: Palabras frecuentes incluyen ltd, tribunal, pty, clr, hca. Observación: Predominan términos relacionados con la aplicación de decisiones y entidades legales específicas, mostrando que los textos describen casos en los que se aplican precedentes o leyes.
- Considered: Palabras frecuentes incluyen mr, evidence, would, made. Observación: Se centra en el análisis y la consideración de pruebas y argumentos, reflejando textos más analíticos y evaluativos de los casos.
- Distinguished: Palabras frecuentes incluyen applicant, application, evidence, decision. Observación: Los textos resaltan diferencias entre casos y la interpretación de solicitudes, mostrando un vocabulario más técnico en la distinción de precedentes y situaciones legales.

En resumen, aunque algunos términos comunes podrían generar cierta confusión entre clases, la presencia de palabras específicas por categoría permite que un modelo basado en embeddings aprenda representaciones diferenciadas y útiles para la tarea de clasificación de textos legales.

2.2.5 Nube de palabras

Finalmente, observaremos las nubes de palabras de cada clase donde las palabras más frecuentes aparecen más grandes, ofreciendo así una visualización más intuitiva y rápida del contenido de cada clase.



Estas visualizaciones validan las explicaciones del anterior apartado.

3. Limpieza y Preparación de Datos

Antes de proceder al entrenamiento de los modelos, se realizó un proceso de preparación de datos para garantizar que los datasets fueran homogéneos y compatibles con el modelo BERT y los adaptadores LoRA. Cabe destacar que, en el análisis descriptivo previo, ya se habían **eliminado textos nulos y se habían estandarizado los nombres de las columnas** para que ambos datasets contasen con las mismas variables: text y label.

3.1 Submuestreo

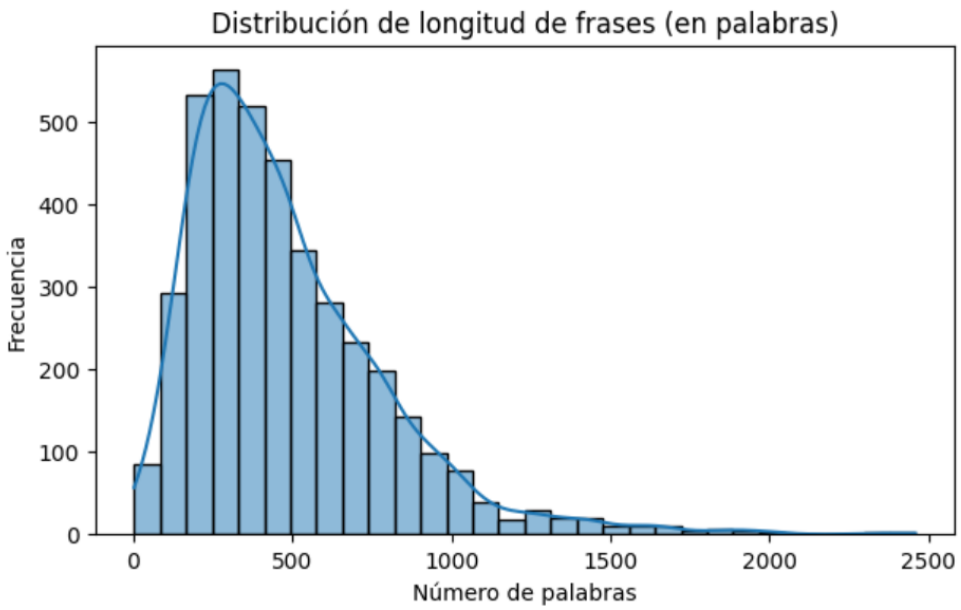
Tal y como se había mencionado en el anterior apartado, para equilibrar la cantidad de ejemplos entre los dominios y facilitar un entrenamiento homogéneo, se aplicó un submuestreo estratificado a ambos datasets:

- **Número de ejemplos finales:** 4.000 por dominio.
- **Estrategia:** se mantuvo la proporción de clases originales mediante muestreo dentro de cada etiqueta (label).

Se verificó además la distribución de la longitud de los textos en palabras, asegurando que los datasets no presentasen sesgos significativos en tamaño de los documentos.

Así pues, obtuvimos las siguientes distribuciones para cada uno de los dominios:

1. <u>Dominio Médico</u>

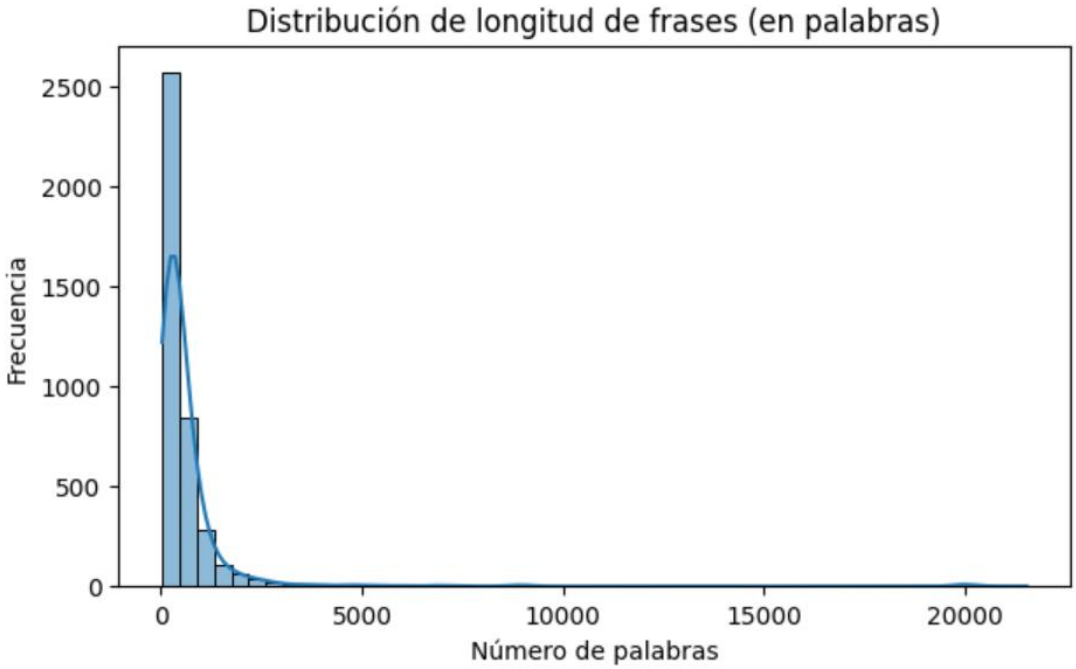



```
Estadísticas de longitud:
count    4000.000000
mean      481.454000
std       309.877354
min         2.000000
25%       259.000000
50%       413.500000
75%       633.000000
max      2460.000000
Name: n_words, dtype: float64
```

Al comparar las estadísticas de esta muestra (4000 ejemplos) con las del dataset original (4999 ejemplos), vemos que son casi idénticas:

- Original (4999): Media=482.3, Mediana=414.0, Std=313.3
- Muestra (4000): Media=481.4, Mediana=413.5, Std=309.9

<u>2. Dominio Legal</u>




```
Estadísticas de longitud:
count     4000.000000
mean       631.595250
std       1509.096776
min         26.000000
25%        181.000000
50%        329.000000
75%        621.000000
max      21535.000000
Name: n_words, dtype: float64
```

Al comparar las estadísticas de esta muestra seleccionada (4000 ejemplos) con las del dataset original (4740 ejemplos), observamos una alta similitud en su distribución de longitud (Media y Desviación Típica casi idénticas), lo que confirma que la muestra es estadísticamente representativa y mantiene las propiedades del conjunto original.

- Original (4740): Media=622.1, Mediana=330.0, Std=1462.1
- Muestra (4000): Media=631.6, Mediana=329.0, Std=1509.1

Conclusiones del Submuestreo

El análisis estadístico presentado confirma que el proceso de submuestreo estratificado ha sido exitoso: las muestras de 4.000 ejemplos mantienen fielmente las propiedades distributivas (media y mediana) de los conjuntos originales, garantizando así la representatividad de los datos para el entrenamiento.

Sin embargo, la comparativa entre dominios hace ver una diferencia estructural importante: mientras que los textos médicos presentan una longitud moderada y

homogénea (Std=310), el corpus legal presenta una variabilidad extrema (Std = 1509) y una longitud media significativamente mayor.

Esta diferencia plantea dos riesgos importantes para el entrenamiento de modelos basados en BERT:

1. **Riesgo de truncamiento:** Dado que el modelo BERT elegido impone un límite estricto de 512 tokens por secuencia, introducir al modelo los textos legales crudos provocaría la pérdida de información relevante ubicada al final de los documentos más extensos.

2. **Sesgo de volumen:** Aunque tengamos el mismo número de documentos (4.000 vs 4.000), el volumen de información real (cantidad de texto) es mucho mayor en el dominio legal, lo que podría desequilibrar el aprendizaje del modelo a favor de este dominio.

Para mitigar estos sesgos y maximizar el aprovechamiento de la información, se hace necesario aplicar en la siguiente fase un mecanismo de segmentación dinámica o Chunking. Esta técnica nos permitirá dividir los textos largos en fragmentos procesables, evitando la pérdida de información por truncamiento y, como veremos posteriormente, equilibrando de forma natural la carga de entrenamiento entre ambos dominios.

3.2 División en entrenamiento y test

Antes de llevar a cabo el mecanismo de Chunking, realizamos la división del dataset en train y test. Este orden evita la fuga de datos que supondría el orden inverso, ya que si hubiéramos realizado el chunking primero:

- El modelo vería la primera mitad de un documento legal en el Train y se le evaluaría con la segunda mitad del mismo documento en el Test.

Así pues, para garantizar la reproducibilidad y la consistencia estadística, la división se ejecutó utilizando la librería scikit-learn bajo los siguientes criterios:

1. **Separación por Dominios:** Se mantuvieron los dataframes médicos y legales separados inicialmente, dado que la estrategia de Fusión de Expertos requiere entrenar adaptadores independientes en una primera fase.

2. **Proporción 80/20:** Se destinó el 80% de los documentos al entrenamiento y el 20% restante a la evaluación (Test), (~4.000 ejemplos).

3. **Estratificación (stratify):** Se utilizó un muestreo estratificado basado en la columna label. Esto asegura que la distribución de clases sea idéntica en el conjunto de entrenamiento y en el de prueba, evitando que clases minoritarias tengan poca representación en la evaluación.

```
Train Médico: 3200 | Test Médico: 800
Train Legal: 3200 | Test Legal: 800
```

Con esta estructura, disponemos de 3.200 documentos por dominio para entrenar a los expertos (o 6.400 al generalista) y 800 documentos por dominio para la evaluación final.

3.3 Chunking

En base a las conclusiones obtenidas en el apartado de Submuestreo, procedemos a implementar un mecanismo de Segmentación o Chunking, siguiendo una estrategia de **Segmentación por Ventana Deslizante.** En lugar de recortar el texto, este método divide cada documento largo en múltiples fragmentos más pequeños que conservan la etiqueta original del documento padre.

Para llevar a cabo esta estrategia, fue necesario configurar el tokenizador de manera específica: se procesaron los textos en modo crudo (truncation=False y add_special_tokens=False), convirtiendo el documento completo a IDs numéricos sin cortes automáticos. Esto permitió inyectar manualmente los tokens especiales de BERT ([CLS] y [SEP]) en cada fragmento individual después de la división, asegurando que cada chunk fuera interpretado por el modelo como una oración completa.

El algoritmo implementado siguió la siguiente lógica:

1. **Tamaño del Bloque (Chunk Size) = 510 tokens:** Se reservan 510 espacios para el contenido real, dejando los 2 espacios restantes para los tokens especiales obligatorios que añade el tokenizador posteriormente.
2. **Solapamiento (Overlap) = 50 tokens:** Se definió un margen de solapamiento entre un fragmento y el siguiente. Esto es importante para evitar cortar una frase justo en el límite de la división. El solapamiento garantiza que el contexto de frontera se preserve en al menos uno de los chunks.
3. **Paso (Stride) = 460 tokens:** El avance de la ventana se calcula como

   Tamaño - Solapamiento.
4. **Filtrado de Ruido:** Se descartaron aquellos chunks resultantes con una longitud inferior a 30 tokens, asumiendo que fragmentos tan cortos (usualmente al final de un documento) carecen de contexto semántico suficiente para ser clasificados y solo introducen ruido.

Los datos finales tokenizados para el entrenamiento son:

- **Chunks Médicos (tokenized_med):** 6.634
- **Chunks Legales (tokenized_leg):** 6.943
- **Total Generalista (tokenized_general_train):** 13.577 (Es el dataset resultante de combinar ambos datasets médico y legal en uno todo junto)

Interpretación de los Datos: Se observa una proporción final de **48,8% (Médico) frente a 51,2% (Legal)**.

Si bien partíamos de un número idéntico de documentos (3.200 de entrenamiento por dominio), la superior extensión de los textos legales provocó que estos generaran más chunks por documento que los textos médicos.

Gracias a este mecanismo, se ha transformado un dataset desbalanceado en volumen de texto (donde lo legal dominaba por longitud) en un dataset de entrenamiento equilibrado en número de ejemplos procesables. Esto garantiza que, durante la fase de Masked Language Modeling (MLM) y Clasificación, el modelo recibirá una cantidad de actualizaciones de gradiente casi simétrica para ambos dominios, eliminando sesgos.

Observación

En este proyecto no se aplicaron procesos adicionales de limpieza de texto ni normalización (como eliminación de stopwords o lematización) porque:

- El tokenizador del modelo BERT elegido ya convierte automáticamente los textos a minúsculas y maneja los subtokens mediante WordPiece, garantizando compatibilidad con el vocabulario del modelo.
- Las técnicas de eliminación de stopwords o lematización, a pesar de aportar mejoras computacionales, pueden limitar el rendimiento del modelo a la hora de capturar contexto sintáctico en dominios como el médico y legal donde pueden el lema de ciertas palabras o algunas stopwords pueden cambiar completamente el significado (sentencias, diagnósticos, etc)
- La eliminación de duplicados o valores nulos ya se realizó durante el análisis descriptivo.

4. Modelado

En este apartado se describe la arquitectura del sistema y la metodología de entrenamiento diseñada para validar la hipótesis.

4.1. Estrategia Global: Pipeline de Dos Etapas (DAPT + SFT)

Con el objetivo de dotar al modelo tanto de capacidades contextuales en los dos dominios (médico y legal) como discriminativas en tareas de clasificación multitarea, se ha diseñado un flujo de trabajo dividido en dos etapas bien diferenciadas. Esta metodología se conoce como **TAPT/DAPT** (Task/Domain-Adaptive PreTraining):

1. Fase 1: Preentrenamiento Adaptativo al Dominio (DAPT - No Supervisado):

En esta etapa, el objetivo no es clasificar, sino adaptar el modelo al lenguaje técnico específico de cada dominio. Se utiliza el objetivo de aprendizaje Masked Language Modeling (MLM), donde el modelo debe predecir palabras ocultas en una oración basándose en el contexto.

- Propósito: Ajustar las distribuciones internas del modelo base para que adquiera familiaridad con términos y expresiones propias de cada dominio antes de enfrentarse a una tarea supervisada.
- Innovación: Como aspecto diferencial, en la parte experimental esta fase se realiza de manera independiente para cada dominio, dando lugar a modelos especializados o expertos modulares. Cada experto aprende las características lingüísticas de su dominio sin interferir con el otro.

2. Fase 2: Ajuste Fino Supervisado (SFT - Supervisado):

Una vez que el modelo ha internalizado el vocabulario, se añade una capa densa final inicializada aleatoriamente, que convierte las representaciones aprendidas en predicciones de clase. Se entrena el sistema completo utilizando las etiquetas de clase del dataset (7 categorías, 4 médicas y 3 legales).

- Propósito: Transformar el conocimiento lingüístico adquirido en la fase 1 en capacidad de decisión para asignar la categoría correcta a cada texto.

Observación

La incorporación de una fase previa de Masked Language Modeling (MLM) resulta indispensable para adaptar el espacio vectorial generalista de BERT a la terminología compleja de los dominios médico y legal antes de llevar a cabo la clasificación. A diferencia del ajuste supervisado directo, que proporciona una

señal de aprendizaje escasa (una única etiqueta por documento), el MLM ofrece un aprendizaje más completo al obligar al modelo a reconstruir el 15% del texto, permitiendo la realineación de los embeddings para capturar relaciones semánticas más profundas y específicas.

Esto garantiza que la posterior fusión de expertos integre conocimiento semántico rico y robusto, en lugar de promediar fronteras de decisión superficiales, permitiendo al sistema final tener una base de conocimiento en los dos dominios de forma más eficaz de cara a la tarea supervisada posterior.

4.2. Selección del Modelo Base: BERT

Como modelo base para todo el experimento se ha seleccionado **BERT,** concretamente la versión **bert-base-uncased**.

La elección de BERT frente a modelos generativos más recientes, como GPT o LLaMA, se justifica por varias razones técnicas:

1. <u>Arquitectura Encoder-Only (Bidireccional):</u>

   A diferencia de los modelos decoder-only como GPT, que procesan el texto de izquierda a derecha, BERT analiza la secuencia completa de manera bidireccional. Esto le permite considerar simultáneamente el contexto anterior y posterior de cada palabra, lo que resulta especialmente ventajoso en tareas de comprensión y clasificación, donde es fundamental captar el significado global de frases complejas, como ocurre en textos médicos o legales.

2. <u>Eficiencia y Tamaño (110M Parámetros):</u>

   Esta variante base cuenta con 12 capas de atención y 110 millones de parámetros. Este tamaño representa un equilibrio entre capacidad de representación y coste computacional, permitiendo su entrenamiento en nuestra GPU T4 (16 GB VRAM) de forma eficiente.

3. <u>Tokenización:</u>

   Su tokenizador nos proporciona las ventajas explicadas anteriormente que nos permite delegar la limpieza y normalización de los textos del dataset.

4.3. Mecanismo de Adaptación: LoRA

Dado que reentrenar los 110 millones de parámetros de BERT en cada fase sería igualmente costoso, se utiliza **LoRA (Low-Rank Adaptation)**.

En lugar de actualizar todos los pesos del modelo, congelamos el modelo base y añadimos matrices de rango bajo (r=16) en los módulos de atención (Query y Value). Esto reduce los parámetros entrenables a menos del 1% del total, permitiendo

generar y almacenar múltiples expertos (adaptadores de ~4MB) de manera eficiente.

Además, es la base que nos permite que la fusión del conocimiento médico y legal sea robusta y eficaz.

4.4. Implementación de la Fase 1: Pre-entrenamiento Adaptativo (MLM)

La implementación de esta fase se centra en el entrenamiento no supervisado del modelo con los corpus médico y legal ya tokenizados. A continuación, se detallan los componentes de dicho entrenamiento desarrollado.

4.4.1. Mecanismo de Auto-Supervisión Dinámica

Para realizar el aprendizaje sin etiquetas manuales, utilizamos el DataCollatorForLanguageModeling de la librería Transformers. Este componente actúa como un enmascarador dinámico durante el entrenamiento:

- **Estrategia de Enmascaramiento:** Se estableció una probabilidad de enmascaramiento del **15%** (mlm_probability=0.15), siguiendo el estándar original del paper de BERT.
- **Enmascaramiento Dinámico:** A diferencia de guardar los datos con máscaras fijas en disco, este collator aplica las máscaras dinámicamente en cada época. Esto significa que si el modelo ve la misma frase en distintas épocas es probable que las palabras ocultas sean diferentes, aumentando la robustez del aprendizaje y evitando el sobreajuste.
- **Gestión de Columnas:** En la función train_domain_model, se eliminan explícitamente la columna label. Esto fuerza al modelo a aprender exclusivamente de la estructura del lenguaje (el texto como input_ids generados por el tokenizador), generando sus propias etiquetas de supervisión a partir de los tokens ocultos. (No necesitamos las etiquetas para este entrenamiento)

4.4.2. Configuración de Adaptadores (LoRA)

Tal y como habíamos mencionado, entrenar un modelo de lenguaje completo es costoso, así que utilizamos la librería PEFT para usar adaptadores LoRA. La configuración específica fue:

- **Rango (r=16):** Se eligió un rango intermedio. Un valor de 16 proporciona suficiente capacidad para aprender la terminología compleja médica y legal sin aumentar en exceso el consumo de memoria.

- **Alpha (alpha=32):** El factor de escala se fijó en 32. La regla alpha = 2 x r garantiza una actualización de pesos estable y robusta.
- **Módulos Objetivo:** Se aplicó la adaptación a las matrices de proyección de atención **Query** y **Value**. De esta manera, resulta más eficiente capturar matices semánticos que si adaptáramos las capas feed-forward. (En esas matrices es dónde están las representaciones más ricas)

4.4.3. Entrenamiento

Para el entrenamiento se definió la función train_domain_model, la cual cargaba el modelo BERT en su forma AutoModelForMaskedLM y aplicaba la configuración LoRA que hemos definido para dicho modelo. El entrenamiento se gestionó mediante la clase Trainer de Hugging Face, optimizando los hiperparámetros para la estabilidad en nuestro hardware (GPU T4):

- **Learning Rate (1e-4):** Se utilizó una tasa de aprendizaje superior a la típica de fine-tuning completo (2e-5). Al entrenar solo un pequeño porcentaje de parámetros (LoRA), necesitamos una tasa más agresiva para que los adaptadores converjan rápidamente.
- **Precisión Mixta (fp16=True):** Se activó el entrenamiento en media precisión para reducir el consumo de VRAM a la mitad y acelerar los cálculos tensoriales.
- **Batch Size (16):** Ajustado al máximo permitido por la memoria de 16 GB tras cargar el modelo y los gradientes.
- **Épocas (3):** Se determinó que 3 épocas completas eran suficientes para adaptar el modelo sin provocar catastrophic forgetting del conocimiento base.
- **Data Collator:** Aquí se instancia el mecanismo de enmascaramiento dinámico que habíamos definido anteriormente con todos sus parámetros.

  → En esta función train_domain_model, se elimina inicialmente la columna label. Esto fuerza al modelo a aprender exclusivamente de la estructura del lenguaje (el texto como input_ids generados por el tokenizador), generando sus propias etiquetas de supervisión a partir de los tokens ocultos. (No necesitamos las etiquetas para este entrenamiento)

4.4.4. Ejecución y Métricas

Finalmente se realizaron tres ejecuciones independientes utilizando la misma función base train_domain_model:

1. **Experto Médico:** Entrenado exclusivamente con tokenized_med['train'].

2. **Experto Legal:** Entrenado exclusivamente con tokenized_leg['train'].
3. **Modelo Generalista:** Entrenado con la concatenación de ambos conjuntos (tokenized_general_train).

Para cada ejecución, se implementó una medición de recursos (torch.cuda.reset_peak_memory_stats) que captura el tiempo de cómputo real (wall-clock time) y el pico de memoria VRAM asignada.

Finalmente, hay que destacar que en esta fase solo se guardan los pesos de los adaptadores (model.save_pretrained), generando expertos ligeros (~4-10 MB) que contienen todo el conocimiento del dominio, facilitando la modularidad que habíamos propuesto en nuestra hipótesis para la fusión.

Al final, tenemos un adaptador especializado en el corpus médico, otro en legal y un adaptador especializado en ambos dominios simultáneamente.

4.6. Implementación de la Fase 2: Clasificación Supervisada y Fusión

Por último, esta fase final tiene como objetivo consolidar el conocimiento lingüístico adquirido durante el pre-entrenamiento (MLM) y alinearlo a la tarea discriminativa multitarea final, de forma que podamos comparar definitivamente el rendimiento de los adaptadores especialistas fusionados respecto al adaptador generalista.

4.6.1. Unificación del Espacio de Etiquetas

En primer lugar, necesitamos un dataset conjunto de ambos dominios, pero ya con las etiquetas correspondientes (no solo el texto) y en formato numérico, así que partimos de los datasets tokenized_med['train/test'] y tokenized_leg['train/test'] y los combinamos. Uno de los problemas al combinar datasets independientes es la colisión de índices numéricos. Originalmente, ambos corpus utilizaban etiquetas superpuestas (Médico 0-3 y Legal 0-2). Para resolverlo, implementamos una estrategia de Desplazamiento de Etiquetas:

- **Mapeo Médico:** Se mantuvieron intactas las etiquetas originales (0: Records, 1: Other, 2: Internal, 3: Surgery).
- **Desplazamiento Legal:** Se aplicó un desplazamiento de +4 a las etiquetas legales. De esta forma, el dataset legal pasó a utilizar los índices 4, 5 y 6.
- **Espacio Resultante:** Esto transformó el problema en una clasificación de 7 clases. El modelo debe aprender implícitamente que el rango [0,3]

corresponde al dominio médico y el rango [4,6] al legal, eliminando cualquier ambigüedad durante el entrenamiento supervisado.

- Obtenemos un dataset combinado para train y otro para test.

4.6.2. Fusión Matemática de Expertos

Como estrategia de fusión de los adaptadores especializados médico y legal obtenidos en la fase 1, se utilizó una combinación lineal de los pesos de ambos. Así pues, se definió una función, llamada fuse_lora_weights, que tomaba los dos adaptadores, los fusionaba y guardaba el adaptador final resultante:

- **Ecuación de Fusión**: Se obtuvieron las matrices de pesos de ambos adaptadores. La operación realizada sobre los tensores se define como:

$$W_fused = alpha \times W_med + (1 - alpha) \times W_leg$$

- **Coeficiente Alpha (alpha=0.5):** Se seleccionó un coeficiente de mezcla de 0.5. Esta decisión se basa en los resultados de la fase de Chunking donde obtuvimos un balance casi perfecto de datos (aproximadamente 50% médicos y 50% legales). Asumimos, por tanto, una importancia equiprobable para la contribución de conocimiento de cada experto.

4.6.3. Entrenamiento

Para entrenar los dos adaptadores, fusionado y generalista, se definió la función train_classifier, la cual utilizó una configuración distinta a la fase MLM, de forma que se cargaba el modelo BERT y se aplicaba el adaptador LoRA, pero esta vez en su forma AutoModelForSequenceClassification para el modelo BERT y especificando además:

- **Cabeza de Clasificación:** Se añade una capa lineal densa inicializada aleatoriamente sobre la salida del token [CLS], indicando como tamaño salida el número de etiquetas del dataset combinado con etiquetas numéricas (7).

- **Modo Trainable (is_trainable=True):** Configuramos el adaptador como entrenable. Esto permite que, durante la propagación hacia atrás, los gradientes actualicen tanto la nueva cabeza clasificadora como los pesos de los adaptadores LoRA, alineándolos a la tarea específica.

El entrenamiento se gestionó mediante la clase Trainer de Hugging Face y en cuanto a hiperparámetros, el entrenamiento supervisado se diseñó para ser idéntico en ambas estrategias, garantizando una comparativa justa:

- **Semilla (Seed=42):** Se fijó la semilla en cuatro niveles (Python, NumPy, PyTorch CPU y GPU) para asegurar determinismo de los resultados.
- **Learning Rate (2e-4):** Se mantuvo una tasa agresiva para facilitar la rápida adaptación de la cabeza y el reajuste de los adaptadores, pero sin ser tan agresiva que en el entrenamiento MLM para que el conocimiento adquirido en dicho entrenamiento previo siga teniendo peso.
- **Precisión Mixta (fp16=True):** Se activó el entrenamiento en media precisión para reducir el consumo de VRAM a la mitad y acelerar los cálculos tensoriales.
- **Batch Size (16):** Ajustado al máximo permitido por la memoria de 16 GB tras cargar el modelo y los gradientes.
- **Épocas (5):** Se incrementó ligeramente el número de épocas respecto a la fase MLM para asegurar la convergencia de la nueva capa de clasificación.
- **Métrica de Evaluación (metric_for_best_model="f1"):** Monitorizamos el F1-Score como métrica principal en lugar del Accuracy. En datasets con ligero desbalance o múltiples clases, el F1 ofrece una visión más equilibrada entre precisión y recall.

4.6.4. Integración Final (Merge and Unload)

Al finalizar el entrenamiento, se aplicó el método merge_and_unload().

Esta operación multiplica las matrices de bajo rango (A x B) y suma el resultado permanentemente a los pesos congelados del modelo base BERT.

El resultado es un modelo BERT estándar de 110 millones de parámetros, que ya no depende de PEFT para la inferencia, eliminando la latencia computacional asociada a los adaptadores en tiempo de ejecución.

4.6.5. Ejecución y métricas

Finalmente, se ejecutaron las dos estrategias para obtener los modelos finales listos para la evaluación comparativa:

1. **Fusión:**
    - Se ejecuta primero la función fuse_lora_weights con alpha=0.5 para crear el adaptador fusionado de ambos dominios, tomando como entrada ambos adaptadores individuales médico y legal.
    - Se entrena el adaptador final sobre el dataset combinado y etiquetas numéricas (4.6.1), utilizando el de train para el propio entrenamiento y el de test para validación, ejecutando la función train_classifier, la

cual añade la cabeza de clasificación al modelo, añade el adaptador y entrena de forma supervisada con los hiperparámetros especificados.

2. **Generalista:**
    - Se carga el adaptador modelo_generalista_lora (entrenado con todos los datos en la fase MLM).
    - Se entrena bajo las mismas condiciones exactas que el adaptador de fusión

En cuanto a las métricas, para cada ejecución, se implementó igual que en el entrenamiento MLM una medición de recursos (torch.cuda.reset_peak_memory_stats) que captura el tiempo de cómputo real (wall-clock time) y el pico de memoria VRAM asignada, pudiendo realizar la comparativa computacional.

Adicionalmente a las métricas de eficiencia, se evaluó el rendimiento predictivo del modelo en cada época. Esto se implementó mediante la definición de la función auxiliar compute_metrics, la cual se pasa como argumento al constructor del Trainer.

El flujo de evaluación diseñado realiza las siguientes operaciones sobre el conjunto de test:

1. **Decodificación de Logits:** La función recibe las salidas crudas del modelo (logits) y aplica una operación np.argmax para transformar las probabilidades en predicciones de clase discretas (índices del 0 al 6).
2. **Cálculo de Indicadores:** Se calculan y reportan las siguientes dos métricas:
    - **Accuracy:** Proporciona una visión global del porcentaje total de aciertos del modelo.
    - **F1-Score Ponderado:** Se seleccionó específicamente esta variante en lugar del promedio simple (macro). Al utilizar weighted, el cálculo del F1 tiene en cuenta e número de instancias reales de cada una de las 7 clases.

De este modo, se obtiene una validación preliminar sobre la eficacia de la estrategia de Fusión frente a la Generalista, indicando de forma anticipada cuál es la tendencia de rendimiento de cada uno antes de proceder a la evaluación definitiva sobre el conjunto de prueba.

5. Análisis de resultados

5.1. Análisis del Espacio Latente (Embeddings)

Para comprender cómo la especialización mediante adaptadores LoRA y la posterior fusión han afectado a la representación interna del lenguaje, se ha realizado una proyección de los embeddings (token [CLS]) utilizando la técnica **T-SNE** (t-Distributed Stochastic Neighbor Embedding).

5.1.1. Separación de Dominios (Médico vs. Legal)

En las visualizaciones se observa una **bipolarización clara del espacio latente**. Los documentos pertenecientes al dominio médico (clases 0-3) y al dominio legal (clases 4-6) se agrupan en dos regiones geográficas del gráfico bien diferenciadas.

- **Observación:** Esta separación es esperable debido a la disparidad de vocabulario técnico ("paciente", "síntoma" vs "jurisprudencia", "artículo"). Sin embargo, tras el pre-entrenamiento adaptativo (MLM) con los expertos, los límites entre ambos dominios se vuelven más nítidos, reduciendo el "ruido" o solapamiento en la frontera entre ambos mundos.

5.1.2. Cohesión de Clases y Fronteras Críticas

Al analizar las clases individuales dentro de cada clúster, podemos extraer conclusiones sobre la dificultad de la tarea:

- **Dominio Médico:** Las cuatro clases presentan una cohesión alta. Los puntos están agrupados de forma compacta, lo que explica por qué este dominio alcanza un F1-Score superior al 0.78. El modelo identifica con facilidad las estructuras lingüísticas propias de cada tipo de informe médico.
- **Dominio Legal:** El gráfico muestra un clúster mucho más disperso. Las clases legales (especialmente aquellas relacionadas con la interpretación de sentencias) aparecen más mezcladas entre sí.
    - *Análisis técnico:* Esto refleja lo observado en el análisis descriptivo; la longitud extrema de los textos legales y el solapamiento semántico hacen que el modelo tenga más dificultades para "encapsular" cada intención en una región única del espacio vectorial.

5.1.3. Impacto de la Fusión de Expertos

Lo más destacable de la comparativa es que el **Modelo Fusionado** mantiene la riqueza de los clústeres especializados. A diferencia de lo que podría ocurrir con un

modelo generalista mal entrenado (donde los clústeres podrían diluirse), la suma aritmética de los pesos de los adaptadores ha preservado la estructura de vecindad de cada experto, permitiendo que el clasificador final trabaje sobre un espacio latente bien organizado y altamente discriminativo.

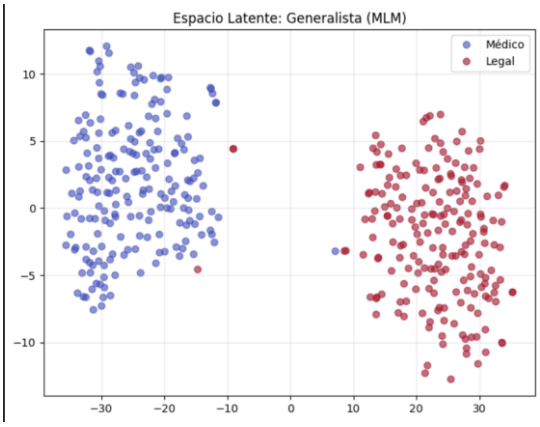
5.1.4. Conclusión e Hipótesis Inicial basada en Embeddings

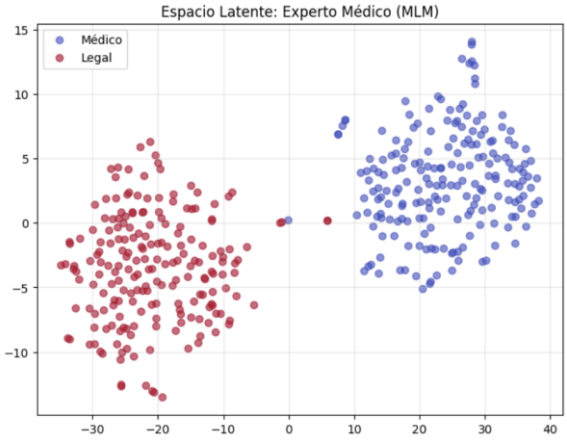
Tras observar la distribución de los datos en el espacio latente, podemos establecer una **hipótesis inicial** sobre cuál de las dos estrategias es más prometedora:

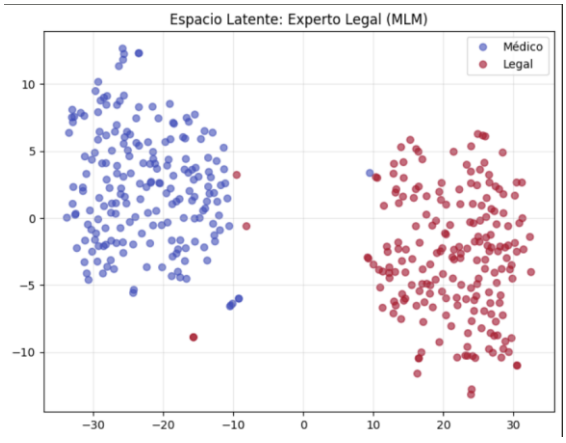
A juzgar por la organización de los clústeres en los gráficos T-SNE, se plantea como hipótesis que el **Modelo de Fusión de Expertos** funcionará mejor que el Modelo Generalista. Esta conclusión preliminar se sustenta en los siguientes puntos:

1. **Preservación de la Identidad Semántica:** Mientras que el modelo generalista se ve obligado a encontrar un "punto medio" lingüístico para procesar ambos tipos de documentos simultáneamente, la arquitectura de expertos permite que cada adaptador capture las sutilezas técnicas de su dominio (médico o legal) de forma pura. En el gráfico, esto se traduce en clústeres más compactos y menos solapados para el modelo fusionado.
2. **Evitación de la "Interferencia de Gradientes":** Durante el entrenamiento del modelo generalista, los patrones semánticos del corpus legal (textos largos y densos) pueden actuar como "ruido" para los patrones médicos (textos breves y directos). La fusión de expertos evita este conflicto al entrenar adaptadores aislados, lo que teóricamente debería resultar en una frontera de decisión más limpia para el clasificador final.
3. **Diferenciación de Dominios:** La clara separación geográfica entre el "hemisferio médico" y el "hemisferio legal" en los embeddings del modelo fusionado sugiere que el sistema ha desarrollado una mayor sensibilidad a la procedencia del texto, lo que facilitará la tarea de clasificación al reducir la ambigüedad entre categorías de distintos dominios.

Por tanto, aunque el modelo generalista es una solución robusta, la **modularidad y especialización** de la fusión de expertos nos lleva a predecir un rendimiento superior, especialmente en la precisión por clase dentro de cada dominio.


Espacio Latente: Generalista (MLM)
Médico
Legal
10
5
0
−5
−10
−30
−20
−10
0
10
20
30


Espacio Latente: Experto Médico (MLM)
Médico
Legal
15
10
5
0
−5
−10
−30
−20
−10
0
10
20
30
40


Espacio Latente: Experto Legal (MLM)
Médico
Legal
10
5
0
−5
−10
−30
−20
−10
0
10
20
30

5.2. Análisis de Resultados del Test y Evaluación Comparativa

Tras el entrenamiento de los clasificadores finales, se procedió a la evaluación sobre un conjunto de test independiente y balanceado que contiene muestras de ambos dominios. Esta fase permite verificar si la especialización previa mediante adaptadores LoRA y su posterior fusión ofrece una ventaja real frente a un enfoque de entrenamiento generalista.

5.2.1. Rendimiento Predictivo

En la siguiente tabla se resumen las métricas de rendimiento obtenidas para ambos modelos. Se ha utilizado el **F1-Score (weighted)** como métrica principal, dada su robustez ante la diversidad de clases, y el **Accuracy** para medir el acierto global.

Ámbito de Evaluación	Métrica	Modelo FUSIONADO	Modelo GENERALISTA	Diferencia
GLOBAL (Mixto)	**F1-Score**	**0.6718**	0.6690	**+0.0028**
Médico (Específico)	**F1-Score**	**0.7881**	0.7873	**+0.0009**
Legal (Específico)	**F1-Score**	**0.5673**	0.5631	**+0.0042**
Global	**Accuracy**	0.6715	**0.6724**	-0.0009

Análisis de las métricas:

- **Superioridad del Modelo Fusionado:** El modelo basado en la fusión de expertos supera al generalista en el F1-Score global. Aunque la diferencia es estrecha (+0.0028), los resultados sugieren que la técnica de promediar pesos de expertos especializados (fusión aritmética) permite capturar mejor las particularidades semánticas de cada dominio sin que uno "contamine" o degrade el aprendizaje del otro.
- **Desequilibrio de Dificultad por Dominio:** Se observa un rendimiento significativamente mayor en el dominio médico (aprox 0.78 F1) frente al legal (aprox 0.56 F1). Esto confirma lo detectado en el análisis descriptivo inicial: los documentos legales, al ser mucho más extensos y densos, presentan una ambigüedad que dificulta la clasificación, independientemente de la estrategia de entrenamiento utilizada.

5.2.2. Eficiencia y Coste Computacional

Un factor crítico en la comparativa es el coste de recursos necesario para alcanzar estos resultados. Gracias al uso de **LoRA**, el entrenamiento ha sido extremadamente eficiente en términos de memoria y tiempo.

Fase de Entrenamiento	Concepto	Expertos Indiv.	M. Generalista
Adaptación (MLM)	Tiempo Total	8.76 min (Méd.) / 9.18 min (Leg.)	17.91 min
	Memoria VRAM	~6.26 GB	6.26 GB
Clasificación Final	Tiempo Total	25.88 min	26.08 min
	Memoria VRAM	3.55 GB	3.57 GB

Observaciones sobre la eficiencia:

1. **Paralelización:** El tiempo total de adaptación del modelo generalista (17.91 min) es equivalente a la suma de los tiempos de los dos expertos. Sin embargo, en un flujo de trabajo real, los expertos podrían entrenarse en paralelo, reduciendo el tiempo de espera a la mitad, una ventaja clave de la arquitectura modular.
2. **Uso de VRAM:** El consumo de memoria se mantuvo constante en torno a los **6.26 GB** durante la fase más pesada (MLM) y bajó a **3.55 GB** en la clasificación. Esto demuestra que la estrategia planteada es viable en hardware de consumo (como la GPU Tesla T4 utilizada), permitiendo trabajar con modelos de la escala de BERT sin requerir infraestructura de alto coste.

5.2.3. Síntesis de Resultados

Los datos confirman la **validez de la Fusión de Expertos**. No solo iguala o supera ligeramente el rendimiento del modelo generalista, sino que lo hace aportando una flexibilidad superior en la arquitectura. Mientras que el modelo generalista requeriría un reentrenamiento completo si se añadiera un tercer dominio (por ejemplo, financiero), la estrategia de expertos permitiría simplemente entrenar un nuevo adaptador liviano y fusionarlo con los existentes.

6. Conclusiones

El presente proyecto ha permitido validar la eficacia de la **Fusión de Expertos LoRA** como una alternativa viable y eficiente frente al entrenamiento generalista tradicional en tareas de clasificación multidominio.

1. **Superioridad de la Fusión:** Los resultados experimentales confirman que la combinación matemática de adaptadores especializados ofrece un rendimiento ligeramente superior (+0.28% F1) al de un modelo entrenado con datos mezclados. Esto sugiere que preservar la especialización de dominios durante la fase de adaptación (MLM) ayuda a generar embeddings más precisos que luego se unifican con éxito para la tarea downstream.
2. **Escalabilidad y Eficiencia:** El uso de **LoRA** ha demostrado ser fundamental para la sostenibilidad del proyecto, permitiendo la adaptación de BERT con un uso de memoria controlado (6.26 GB). La arquitectura de fusión es intrínsecamente más escalable; añadir un tercer dominio (por ejemplo, financiero) solo requeriría entrenar un nuevo adaptador liviano y promediar sus pesos con los existentes.
3. **Sensibilidad al Dominio:** Se ha evidenciado que el rendimiento está fuertemente condicionado por la naturaleza del texto. El dominio legal, con secuencias más largas y complejas, presenta un desafío mayor para la arquitectura BERT-base. Estrategias futuras de *chunking* más agresivas o el uso de modelos con ventanas de contexto más amplias podrían mejorar estos resultados específicos.
4. **Consideraciones Finales:** La técnica de fusión LoRA no solo iguala la precisión de los métodos tradicionales, sino que introduce una flexibilidad arquitectónica crítica para entornos de producción donde los datos de diferentes dominios pueden no estar disponibles simultáneamente o requieren actualizaciones independientes.